

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250123>

Forming of the groups

Visit the BB (Blackboard) to join a group

DL: Tue., 28.01., 12:00 (noon)

Teaching assistants (TAs)

1. Sunil Sharma, sunils@stud.ntnu.no
2. Hooman Hooshmand, hoomanh@stud.ntnu.no

To support students with use of ML tools, methods, APIs.

Weekly time slots with TAs:

Tuesdays: 14:30-16:30, [L-Big Ben](#)

Wednesdays: 14:00-16:00, [L-Big Ben](#)

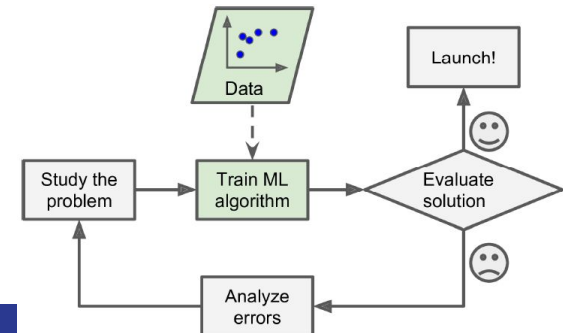
A1: Supervised learning in product design / development (DL 20.02.)

Find a dataset for an engineering product. Check if you can predict product performance based on its technical drawings, 3D models, other digital descriptions and representations (e.g. by using regression techniques) AND/OR find invalid notations in technical drawings (e.g. using classification techniques).

Follow the ML project checklist. **Evaluate and report on each phase of the checklist.** Where applicable, try to replicate and compare any available results with your results received using regression and/or classification techniques you have applied.

Check for datasets here:

- * <https://www.kaggle.com/>
- * <https://decode.mit.edu/projects/eng-datasets/>



Outline : Classification

- MNIST database
- Training binary classifier
- Performance measures
 - Measuring Accuracy Using Cross-Validation
 - Confusion Matrix
 - Precision and Recall
 - Precision/Recall Trade-off
 - The ROC Curve
- Multiclass Classification
- Error Analysis
- Multilabel classification
- Multioutput classification
- Assignment 1

-
- **Source:** Aurelien Geron, “Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems”, 2019 ([Available online](#)) / **Chapters 1 and 3**

Classification (predicting classes)

- In supervised learning, the training set you feed to the algorithm includes the **desired solutions**, called **labels**.
- Classification is a typical supervised learning task. Learns to put input data to a class (correct label).

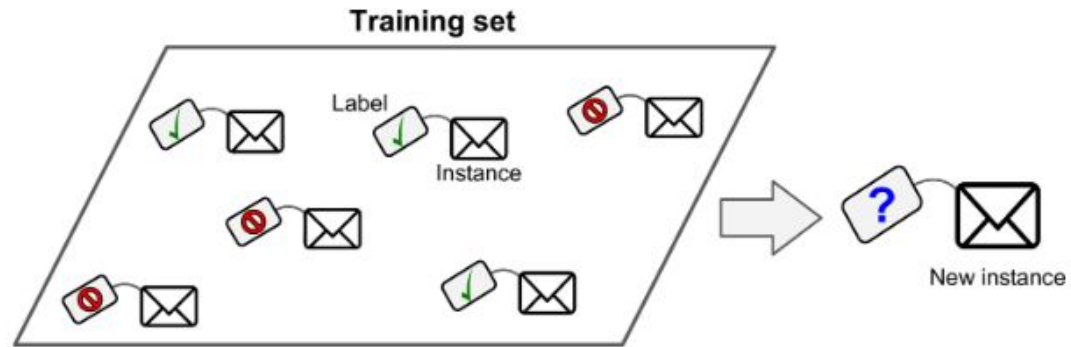


Figure 1-5. A labeled training set for spam classification (an example of supervised learning)

Regression (predicting values)

Another typical task is **to predict a target numeric value**, such as the price of a car, given a set of **features** (mileage, age, brand, etc.) called **predictors**. This sort of task is called **regression** (Figure 1-6). To train the system, you need to give it many examples of cars, including both their predictors and their labels (i.e., their prices).

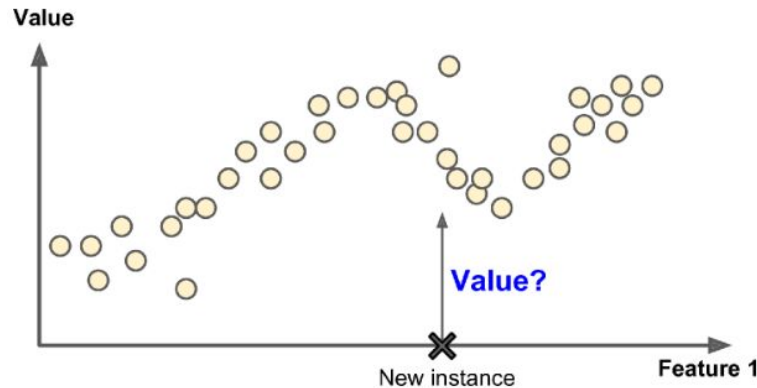


Figure 1-6. A regression problem: predict a value, given an input feature (there are usually multiple input features, and sometimes multiple output values)

Attribute, feature

In ML an attribute is a data type (e.g., “mileage”, “voltage”), while a feature has several meanings, depending on the context, but generally means an attribute plus its value (e.g., “mileage = 15,000”, “V_in = 10”). Many people use the words attribute and feature interchangeably.

MNIST database

Code: [Chapter 3](#)

MNIST database

- A set of 70,000 small images of digits handwritten by high school students and employees of the US Census Bureau.
- Each image is labeled with the digit it represents.
- Used to compare, evaluate classification algorithms

Datasets loaded by Scikit-Learn generally have a similar dictionary structure, including the following:

- A **DESCR** key describing the dataset
- A **data** key containing an array with one row per instance and one column per feature
- A **target** key containing an array with the labels

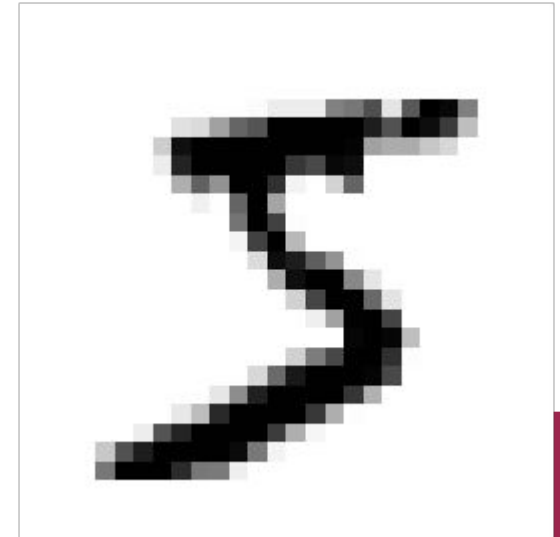


MNIST database (For info: [HERE](#) and [HERE](#))

```
A network error occurred while downloading https://api.openml.org/api/v1/json/data/list/data_name/mnist_784/limit/2/status/active/. Retrying...  
warn(|
```

- [Setup up to step 8, and then 12](#)

28 x 28 pixels with each feature representing pixel intensity, from 0 (white) to 255 (black).



MNIST database

Creating a training set. It is already **shuffled** for us.

Step 13:

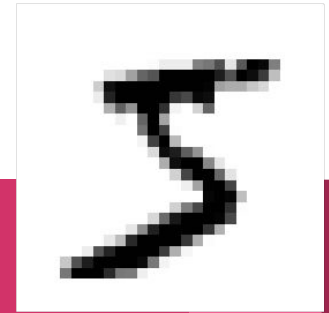
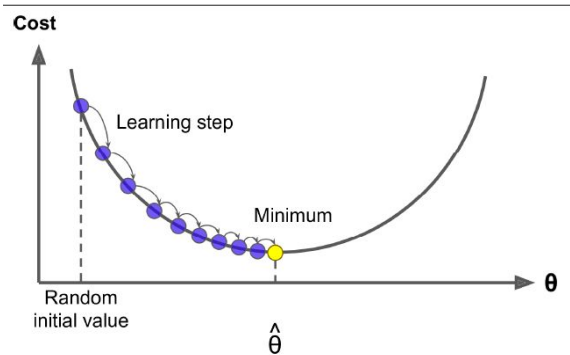
```
X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000],  
y[60000:]
```

Shuffling can be a bad idea in some contexts - e.g. time series data (weather condition, stock market prices).

Training Binary Classifier

Binary Classifier

- Simplifying a problem. Suppose there are just one digit, e.g. 5.
- “5-detector” is a binary classifier. Two classes: **5** and **not 5**.
- Steps 14 - 17
- SGDClassifier



(Stochastic) Gradual Descent

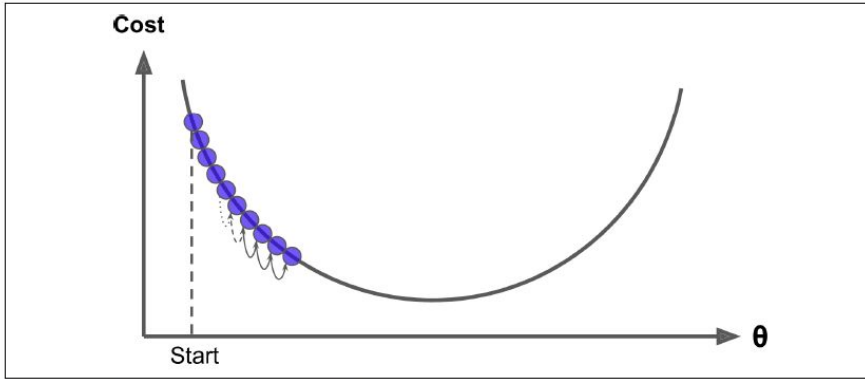


Figure 4-4. Learning rate too small

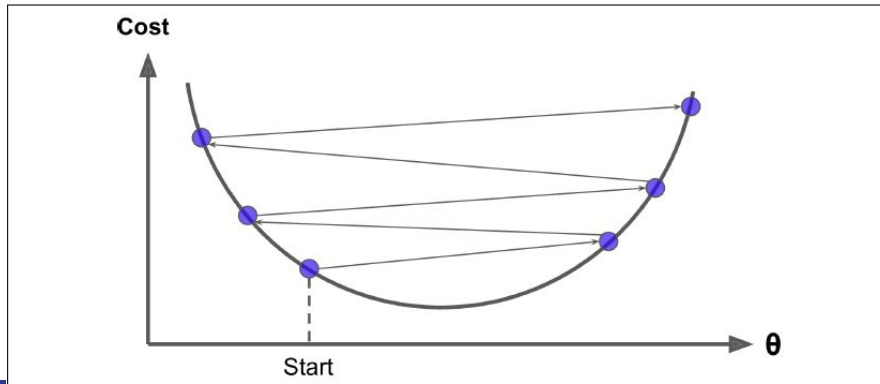


Figure 4-5. Learning rate too large

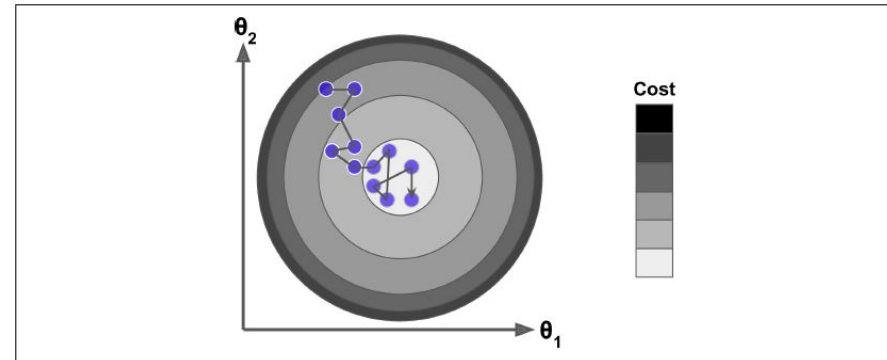


Figure 4-9. Stochastic Gradient Descent

Performance measures

- Measuring Accuracy Using Cross-Validation
- Confusion Matrix
- Precision and Recall
- Precision/Recall Trade-off
- The ROC Curve

Measuring Accuracy Using Cross-Validation (1)

- Accuracy - ratio of correct predictions
- Steps 18 - 20

At each iteration the code creates a clone of the classifier, trains that clone on the training folds, and makes predictions on the test fold. Then it counts the number of correct predictions and outputs the ratio of correct predictions.

cross_val_score() function to evaluate your **SGDClassifier** model

```
>>> from sklearn.model_selection import cross_val_score  
>>> cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy")  
array([0.96355, 0.93795, 0.95615])
```

Measuring Accuracy Using Cross-Validation (2)

Looking at a very dumb classifier that just classifies every single image in the “not-5” class:

```
from sklearn.base import BaseEstimator

class Never5Classifier(BaseEstimator):
    def fit(self, X, y=None):
        pass
    def predict(self, X):
        return np.zeros((len(X), 1), dtype=bool)

>>> never_5_clf = Never5Classifier()
>>> cross_val_score(never_5_clf, X_train, y_train_5, cv=3, scoring="accuracy")
array([0.91125, 0.90855, 0.90915])
```

Over 90% accuracy, but only about 10% of the images are 5s, so if you always guess that an image is not a 5, you will be right about 90% of the time.

Measuring Accuracy Using Cross-Validation (3)

Accuracy (ratio of correct predictions) is generally not the preferred performance measure for classifiers, especially when you are dealing with *skewed datasets* (i.e., when *some classes are much more frequent than others*).

Confusion Matrix (1)

- A better idea for evaluating the performance of a classifier is to count the number of times instances of class A are classified as class B.
- In given example of numbers: to know the number of times the classifier confused images of 5s with 3s - look in the 5th row and 3rd column of the **confusion matrix**.
- [Steps 21-23](#)

Confusion Matrix (2)

Actual

	Predicted									
	1	2	3	4	5	6	7	8	9	0
1										
3										
3										
4										
5										
6										
7										
8										
9										
0										

Confusion Matrix (3)

In addition, “accuracy of the positive predictions” - *precision* of the classifier.

Equation 3-1. Precision

$$\text{precision} = \frac{TP}{TP + FP}$$

TP - number of true positives, *FP* - number of false positives.

Precision used along *recall* or *sensitivity* or *true positive rate*, where *FN* - the number of false negatives:

Equation 3-2. Recall

$$\text{recall} = \frac{TP}{TP + FN}$$

Confusion Matrix (4)

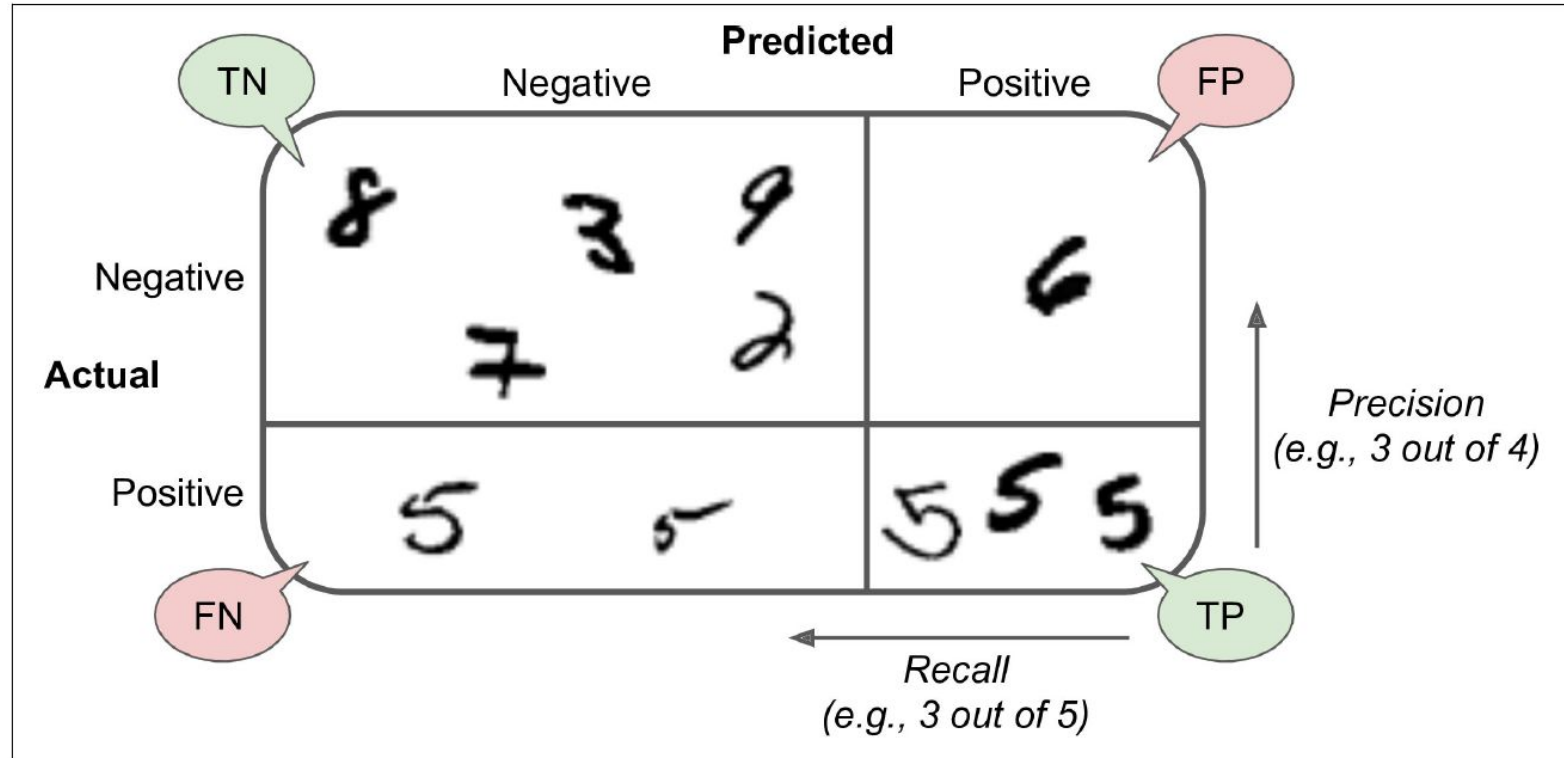


Figure 3-2. An illustrated confusion matrix

Precision and Recall (1)

- Steps 24 - 29

```
>>> from sklearn.metrics import precision_score, recall_score  
>>> precision_score(y_train_5, y_train_pred) # == 4096 / (4096 + 1522)  
0.7290850836596654  
>>> recall_score(y_train_5, y_train_pred) # == 4096 / (4096 + 1325)  
0.7555801512636044
```

with this the 5-detector, when claiming an image to be 5, it is correct 72,9 % (precision).
It only detects 75,6 % of the 5s (recall).

Precision and Recall (2)

Combining precision and recall into single metric - the F_1 score.

Equation 3-3. F_1

$$F_1 = \frac{2}{\frac{1}{\text{precision}} + \frac{1}{\text{recall}}} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{TP}{TP + \frac{FN + FP}{2}}$$

The F_1 score favors classifiers that have similar precision and recall. This is not always what you want: in some contexts you mostly care about precision, and in other contexts you really care about recall:

“high” precision, “low” recall - safe videos for kids: some good videos may not get through, but neither the bad ones.

“low” precision, “high” recall - shoplifters: security may get false alarms, but catching most shoplifters.

Precision and Recall (3)

Impossible to have both. Increasing *precision* reduces *recall*, and vice versa. This is called the *precision/recall tradeoff*.

Precision/Recall Trade-off (1)

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

- Steps 30 - 43

(Fine tuning SGDClassifier with a threshold).

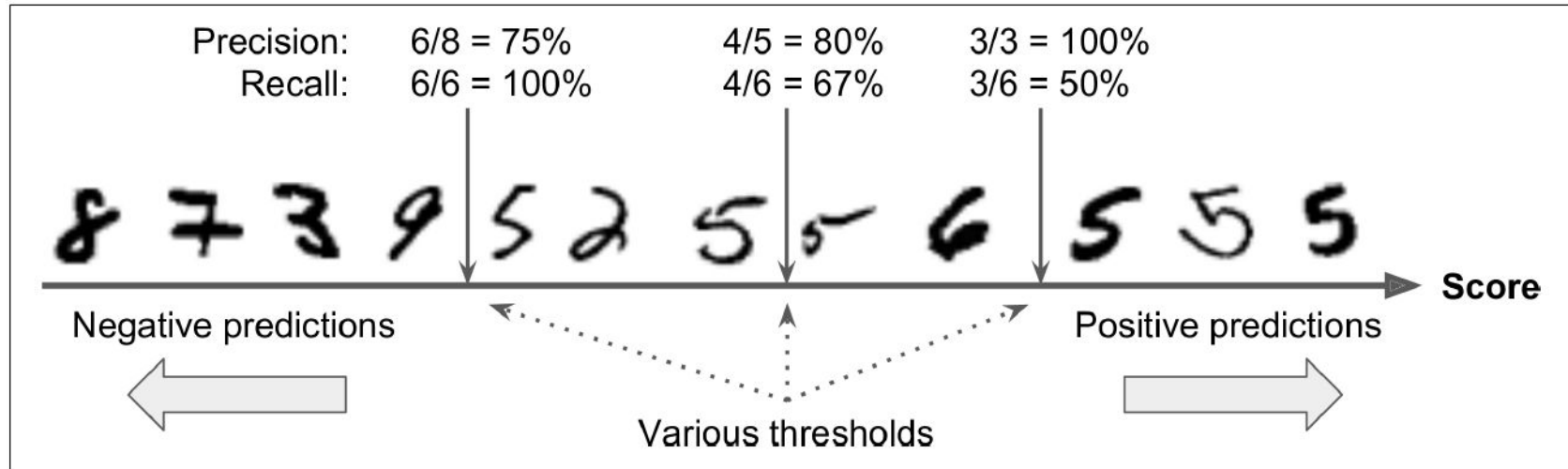


Figure 3-3. Decision threshold and precision/recall tradeoff

Precision/Recall Trade-off (2)

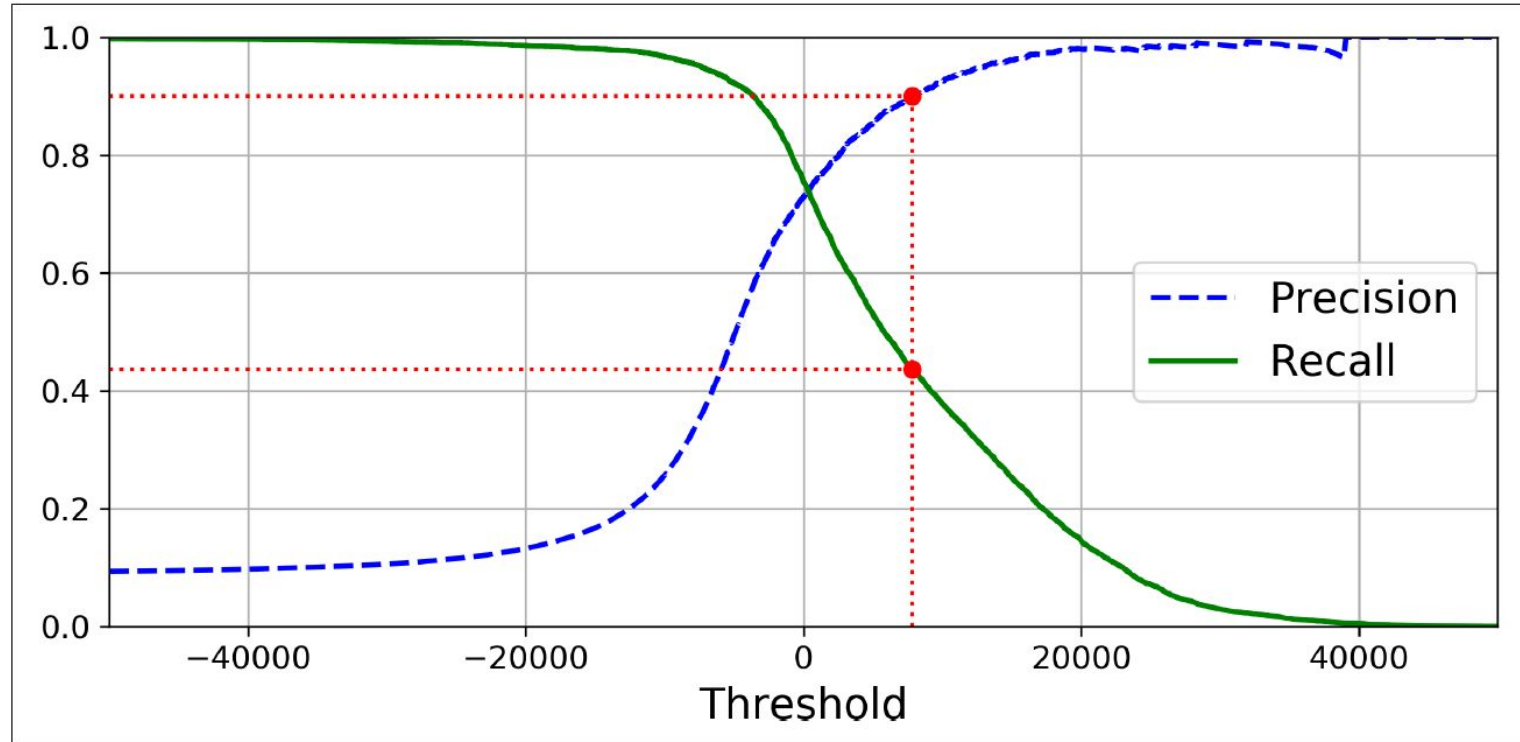


Figure 3-4. Precision and recall versus the decision threshold

Precision/Recall Trade-off (3)

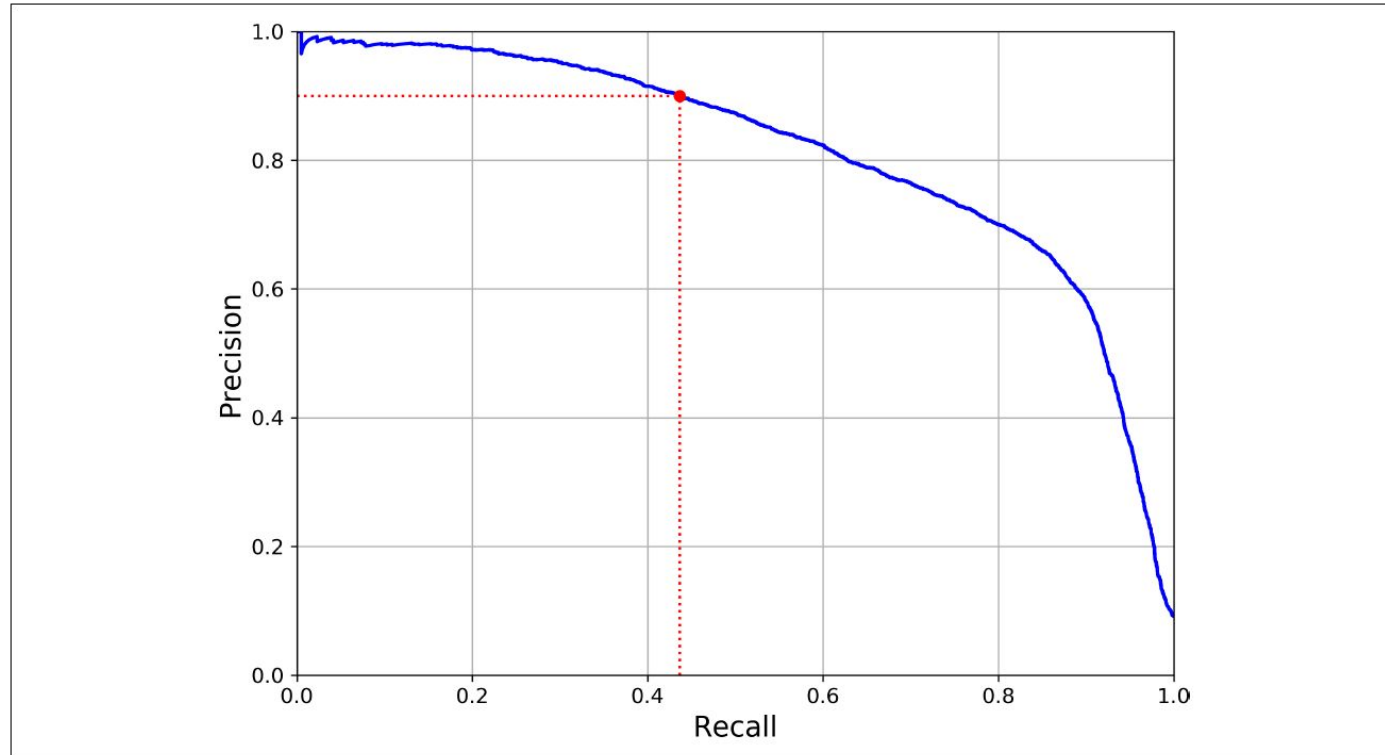


Figure 3-5. Precision versus recall

Precision/Recall Trade-off (4)

You may create a classifier with virtually any precision. However, may not be very useful if recall is low.

The ROC Curve (1)

Another common tool with binary classifiers: The **receiver operating characteristic** (ROC). Similar to the precision/recall curve, but plots *true positive rate* (recall) against the *false positive rate* (FPR).

FRP: the ratio of negative instances that are incorrectly classified as positive.

Steps 44 - 52

The ROC Curve (2)

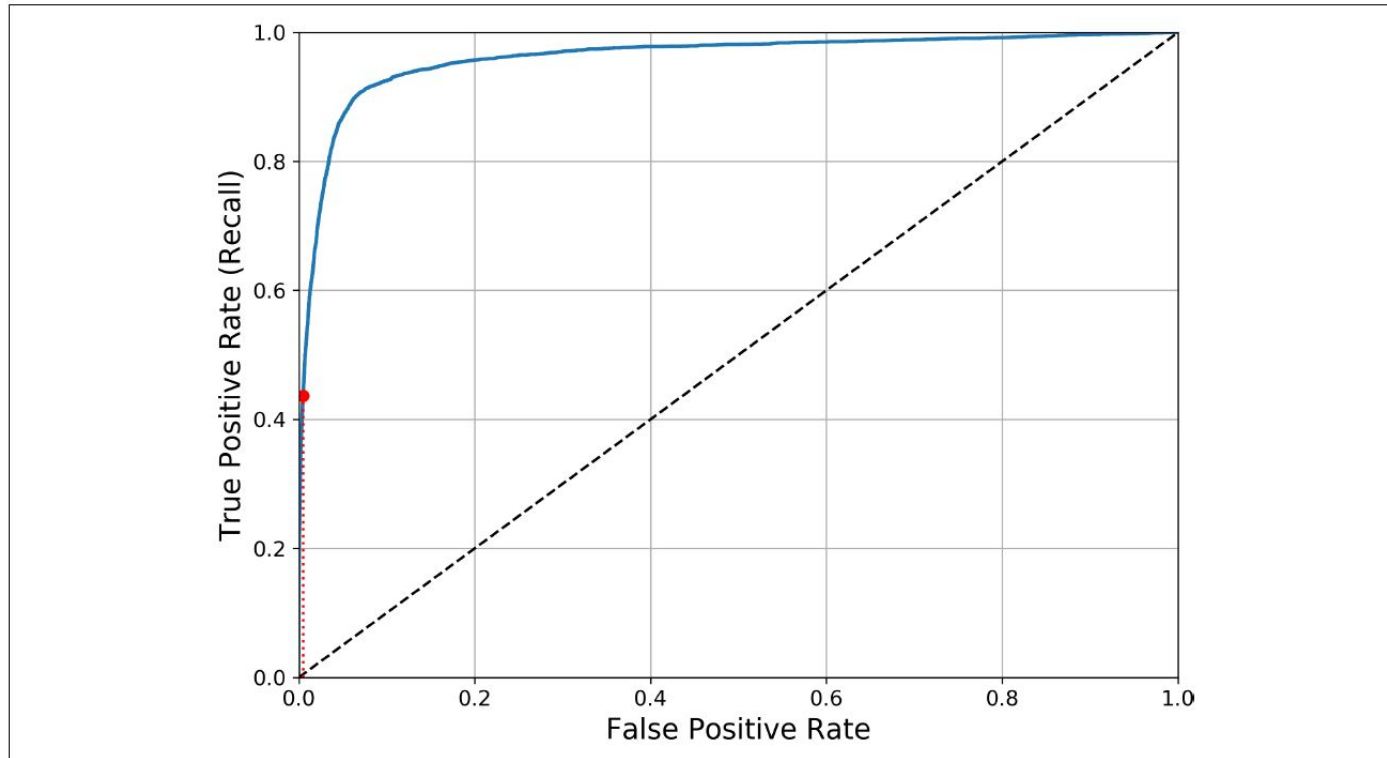


Figure 3-6. ROC curve

The ROC Curve (3)

RandomForest vs.SGD

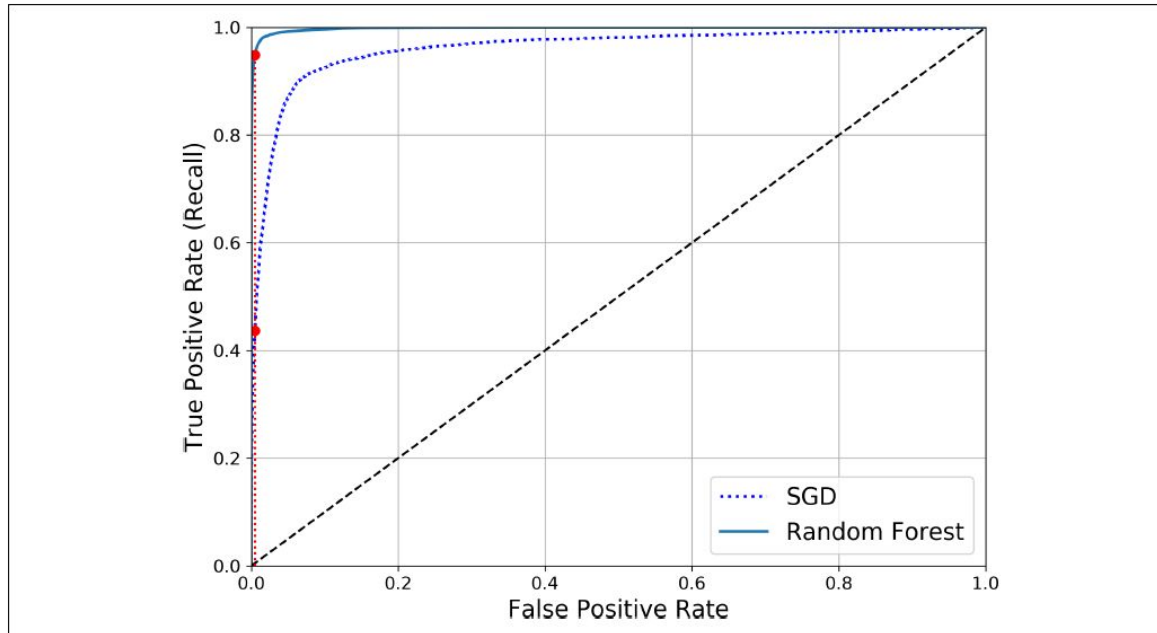


Figure 3-7. Comparing ROC curves

Multiclass Classification

Steps 53 - 63

Multiclass Classification (1)

Whereas binary classifiers distinguish between two classes, multiclass classifiers (also called multinomial classifiers) **can distinguish between more than two classes**.

Some algorithms (such as Random Forest classifiers or naive Bayes classifiers) are capable of handling multiple classes directly. Others (such as Support Vector Machine classifiers or Linear classifiers) are strictly binary classifiers.

Various strategies that you can use **to perform multiclass classification using multiple binary classifiers**.

Multiclass Classification (2)

one-versus-all (OvA) strategy (also called one-versus-the-rest): 0-detector, 1-detector, 2-detector... : The classifier, which outputs the highest score.

one-versus-one (OvO) strategy: a binary classifier for every pair. If N classes, then to train $N \times (N - 1) / 2$ classifiers. the MNIST problem $\rightarrow$ 45 binary classifiers. Run image through 45 classifiers to see which one wins the most “duels”. (In OvO, each classifier is trained on the part of the training set it must distinguish).

Error analysis

Steps 64 - 69

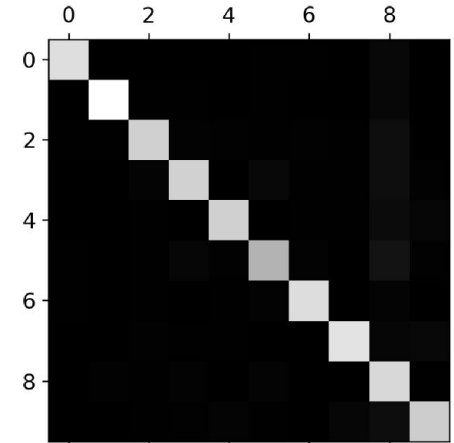
Error analysis (1)

In a real project, you would follow the steps in your Machine Learning project checklist: exploring data preparation options, trying out multiple models, shortlisting the best ones and fine-tuning their hyperparameters using GridSearchCV, and automating as much as possible.

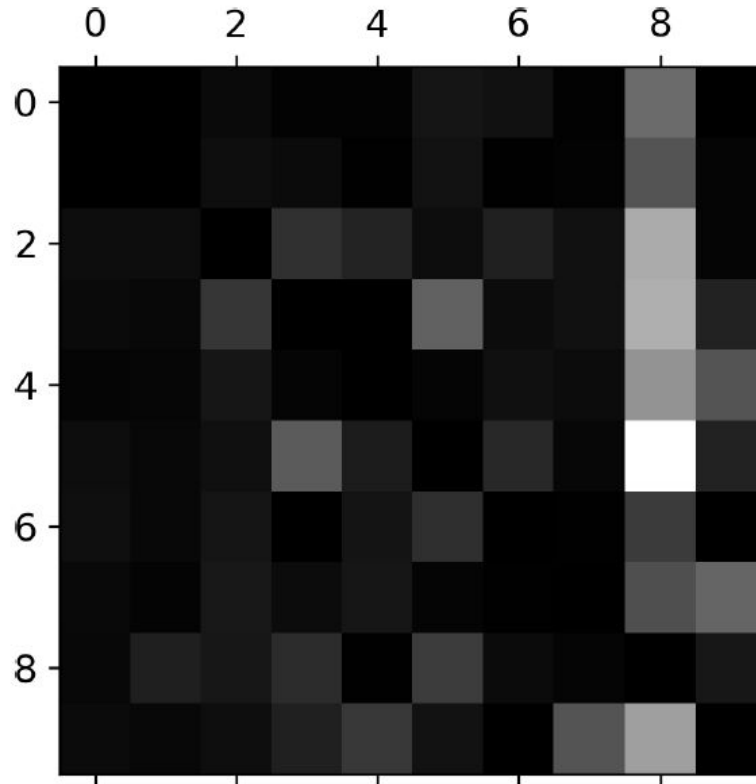
Assume that we have found a promising model and you want to find ways to improve it.

Error analysis: Confusion matrix

```
array([[5578,    0,   22,    7,    8,   45,   35,    5,  222,    1],
       [    0, 6410,   35,   26,    4,   44,    4,    8,  198,   13],
       [  28,   27, 5232,  100,   74,   27,   68,   37,  354,   11],
       [  23,   18,  115, 5254,    2,  209,   26,   38,  373,   73],
       [  11,   14,   45,   12, 5219,   11,   33,   26,  299,  172],
       [  26,   16,   31,  173,   54, 4484,   76,   14,  482,   65],
       [  31,   17,   45,    2,   42,   98, 5556,    3,  123,    1],
       [  20,   10,   53,   27,   50,   13,    3, 5696,  173,  220],
       [  17,   64,   47,   91,    3,  125,   24,   11, 5421,   48],
       [  24,   18,   29,   67,  116,   39,    1,  174,  329, 5152]])
```



Error analysis: Plotting errors



Error analysis: Analysing individual errors

The classifier
is quite sensitive
in this case.
Preprocess?

wrong

classified as 3	classified as 5
3 3 3 3 3	3 3 3 3 3
3 3 3 3 3	3 3 3 3 3
3 3 3 3 3	3 3 3 3 3
3 3 3 3 3	3 3 3 3 3
3 3 3 3 3	3 3 3 3 3
5 5 5 5 5	5 5 5 5 5
5 5 5 5 5	5 5 5 5 5
5 5 5 5 5	5 5 5 5 5
5 5 5 5 5	5 5 5 5 5
5 5 5 5 5	5 5 5 5 5

Multilabel classification

Steps 70 - 72

Multilabel classification

Assigning multiple classes.

People in the same picture. Elements in the same drawing. **Large AND odd numbers.**

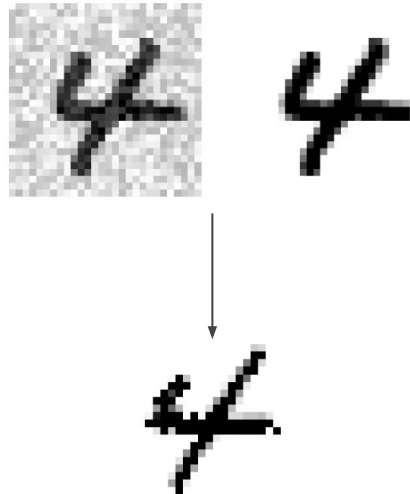
Multiclass classification

Steps 73 - 75

Multiooutput classification

Generalization of multiclass classification, where each label can be multiclass (having more than two possible values).

Noise removal from the picture. Classifier's output is multilabel (one label per pixel) and each label can have multiple values (pixel intensity).



Takeaways

- Metrics for classification tasks,
- Selecting appropriate precision/recall tradeoff,
- Compare classifiers,
- OvR vs OvO strategies for multiclass classification.

A1: Supervised learning in product design / development (DL 20.02.)

Find a dataset for an engineering product. Check if you can predict product performance based on its technical drawings, 3D models, other digital descriptions and representations (e.g. by using regression techniques) AND/OR find invalid notations in technical drawings (e.g. using classification techniques).

Follow the ML project checklist. **Evaluate and report on each phase of the checklist.** Where applicable, try to replicate and compare any available results with your results received using regression and/or classification techniques you have applied.

Check for datasets here:

- * <https://www.kaggle.com/>
- * <https://decode.mit.edu/projects/eng-datasets/>

